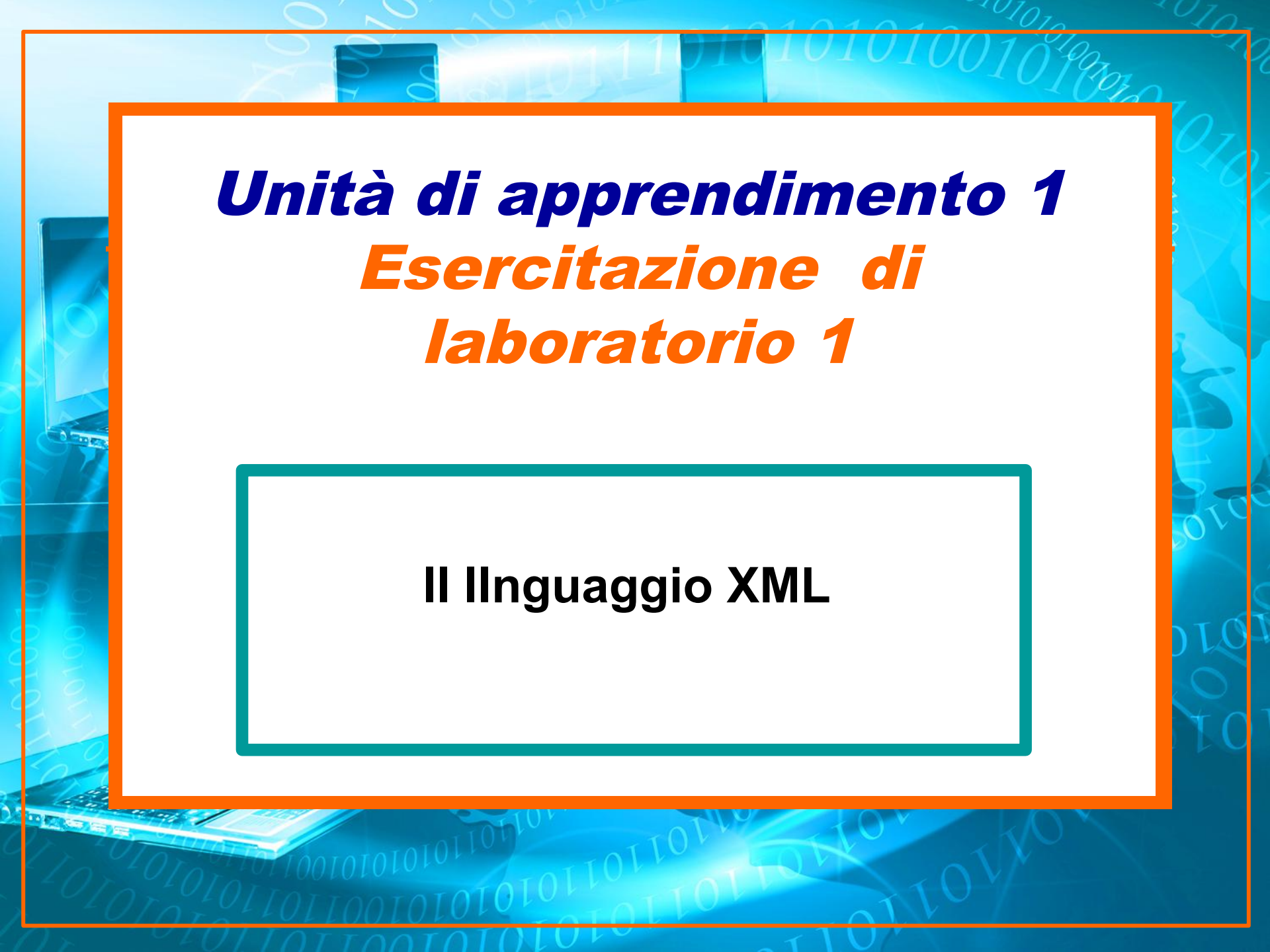


Unità di apprendimento 1

**Architettura di rete e formati
per lo scambio dei dati**



The background of the slide is a blue gradient with a pattern of binary code (0s and 1s) in a lighter blue color. On the left side, there is a partial view of a laptop screen and keyboard.

Unità di apprendimento 1

Esercitazione di laboratorio 1

Il linguaggio XML

Generalità

- L'**eXtensible Markup Language (XML)** non è un linguaggio di markup né l'evoluzione HTML
- È un **metalinguaggio di markup**, cioè un linguaggio che permette di definire **altri** linguaggi di markup (dal greco “meta” che significa oltre o al di là)
- XML non ha tag predefiniti e non serve né per programmare né per definire pagine web
- È costituito da un insieme standard di **regole** sintattiche per modellare la struttura di documenti e dati
- È diventato uno standard per lo scambio dei dati di tipo commerciale via Internet

XML (1)

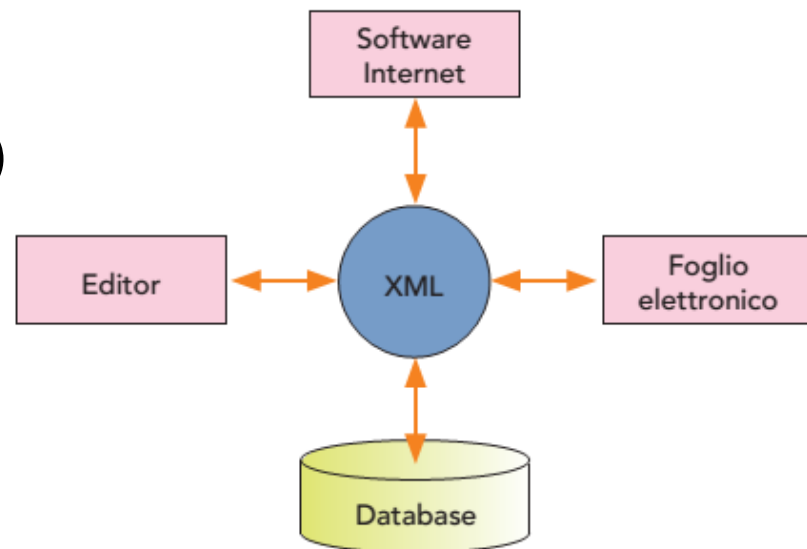
- Le specifiche ufficiali di XML sono state definite dal W3C (<https://www.w3.org/TR/xml/>)
- Essendo un documento di testo può essere letto e modificato con qualsiasi editor di testo, quindi è compatibile con qualsiasi sistema HW e SW
- Le regole formali su cui si basa indicano esattamente come comporre un documento XML
- HTML si occupa di visualizzare i dati, XML di **descrivere** la natura dei dati
- XML ha lo scopo di rappresentare contenuti testuali organizzati in modo **gerarchico**

XML (2)

- Ciò significa che i file XML possono essere trasmessi tra aziende e utenti diversi, senza **alcun vincolo**
 - hanno come suffisso **.XML**
 - consentono di separare i dati dalla rappresentazione grafica
 - possiedono un insieme **unico** di regole di sintassi che consentono di leggere e scrivere i contenuti rappresentati
 - sono basati su tag del tipo **<nome></nome>** che permettono di leggere le informazioni in essi contenute, in base al loro nome e **non necessariamente** alla loro posizione
 - a ogni documento XML può essere associata la definizione della **struttura** utilizzata per renderlo facilmente leggibile e per rendere rapido il processo di **verifica** della struttura del documento

Utilizzo dell'XML (1)

- Con l'XML i dati strutturati possono essere memorizzati all'interno delle pagine HTML
- Possiamo allora usare l'HTML solo per il layout e la presentazione dei dati
- Questa modalità di sviluppo avviene definendo le cosiddette aree **data islands** (isole dei dati)
- Ne consegue che se i dati cambiano non c'è bisogno di cambiare l'HTML



Utilizzo dell'XML (2)

- **Scambio** di dati con l'XML
 - XML risolve problemi di incompatibilità tra sistemi diversi, basta convertire i dati in formato XML
- **Condivisione** dei dati
 - essendo puro testo, l'XML fornisce un modo di condividere i dati slegato sia dal software sia dall'hardware
- **Memorizzazione** dei dati
 - può essere utilizzato per memorizzare dati in un file o in un database

La sintassi XML (1)

- Essendo un file di testo abbiamo detto che XML è leggibile su qualsiasi piattaforma
- Un documento XML è costituito da due parti:
 - un **prologo**, costituito dalla dichiarazione XML e il riferimento (opzionale) ad altri documenti che ne definiscono la struttura
 - un **corpo**, documento XML vero e proprio

La sintassi XML (2)

- La prima riga del prologo (opzionale) viene chiamata **XML declaration**

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

Versione di XML

Codifica
dei caratteri

Indica se il documento è
conforme a una sintassi
esterna (standalone="no")

la versione 1.0 è quella più utilizzata (esiste anche la 1.1, ma non è ampiamente supportata)

- La seconda riga del prologo (opzionale) contiene indicazioni a documenti esterni

```
<?xml-stylesheet type="text/css" href="greco.css"?>  
<!DOCTYPE prodotti SYSTEM "prodotti.dtd">
```

istruzioni di elaborazione (Processing instruction)

istruzioni di validazione (Doctype declaration)

La sintassi XML (3)

- Il contenuto del documento deve corrispondere a una **gerarchia** di tag non sovrapposti
- quindi i tag che si aprono per ultimi devono chiudersi per primi e ogni tag deve avere una chiusura

```
<pagina>
<titolo>esempio di documento in stile XML</titolo>
  <testo>
    <paragrafo>testo testo testo</paragrafo>
    <paragrafo>testo testo testo</paragrafo>
  </testo>
</pagina>
```

La sintassi XML (4)

- È possibile inserire commenti liberi racchiudendoli dentro i tag
`<!-- tuo commento -->`
- Deve sempre esserci un tag che racchiuda tutti gli altri, che viene chiamato **root** (radice)
- Inoltre l'XML è **case sensitive**, pertanto un tag maiuscolo è diverso dallo stesso tag scritto in minuscolo!

La sintassi XML (5)

- Un documento XML si dice **ben formato** se rispetta le regole sintattiche
- Un documento XML si dice **valido** se rispetta le regole sintattiche e semantiche
- Regole sintattiche:
 - **XML declaration** corretta
 - un unico elemento **radice**
 - una corretta **nidificazione** degli elementi
 - ogni elemento deve avere un tag di **apertura** e di **chiusura** (se vuoto si può usare la forma abbreviata <nometag/>)
 - i **nomi** dei tag di apertura e chiusura devono coincidere
 - gli attributi sono sempre racchiusi tra **singoli** e **doppi apici**

La sintassi XML (5)

- i nomi degli elementi sono **case sensitive**
- i nomi degli elementi devono iniziare con una lettera o un trattino basso
- i nomi degli elementi non possono iniziare con le lettere xml (o XML, o Xml, ecc.)
- i nomi degli elementi possono contenere lettere, cifre, trattini, trattini bassi e punti, **ma usate il buon senso e seguite le regole sintattiche dei linguaggi di programmazione che conoscete (Python, C#, Java, ...)**
- i nomi degli elementi non possono contenere **spazi**
- gli attributi sono sempre racchiusi tra **singoli** e **doppi apici**

La sintassi XML (6)

ESEMPIO *Documento XML per definire una rubrica*

```
<?xml version="1.0" encoding="ISO-8859-1"?>
- <rubrica>
  - <persona>
    <nome>Paolo</nome>
    <cognome>Camagni</cognome>
    <telefono_casa>0288888888</telefono_casa>
    <telefono_lavoro>0299999999</telefono_lavoro>
    <indirizzo_email>paolo@camagni.it</indirizzo_email>
  </persona>
  - <persona>
    <nome>Riccardo</nome>
    <cognome>Nikolassy</cognome>
    <telefono_casa>0277777777</telefono_casa>
    <telefono_lavoro>0299999999</telefono_lavoro>
    <indirizzo_email>riccardo@nikolassy.it</indirizzo_email>
  </persona>
</rubrica>
```

Elementi dell'XML

- Gli elementi di XML sono liberi ovvero non vi sono limitazioni sui nomi dei tag se non quelle sintattiche (anche se è buona norma rispettare anche quelle semantiche), si possono scegliere liberamente i nomi dei tag

ESEMPIO *Messaggio tra due utenti*

```
<?xml version="1.0" encoding="ISO-8859-1"?>
- <messaggio>
    <mittente>Mario</mittente>
    <destinatario>Maria</destinatario>
    <oggetto>promemoria</oggetto>
    <testo>ricordati che siamo d'accordo di vederci venerdì sera </testo>
</messaggio>

<?xml version="1.0" encoding="ISO-8859-1"?>
- <messaggio>
    <data>2017-07-12</data>
    <mittente>Mario</mittente>
    <destinatario>Maria</destinatario>
    <oggetto>promemoria</oggetto>
    <testo>ricordati che siamo d'accordo di vederci venerdì sera </testo>
</messaggio>
```

La struttura di un documento XML: XSD (1)

- Ma come definiamo le regole sintattiche di un documento XML? Come verifichiamo se i tipi dei dati sono rispettati?
- Il primo formato standard per la definizione della struttura di un documento XML è stato **DTD**, Document Type Definition
- DTD presentava molte limitazioni tra cui una sintassi non compatibile con XML
- Successivamente si è introdotto **XSD**, XML Schema Definition
- Il linguaggio usato negli XSD invece è XML!
- Ogni XSD deve avere un elemento radice denominato **schema** con un attributo predefinito che rimanda agli elementi e ai tipi di dati standard utilizzabili

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```


La struttura di un documento XML: XSD (2)

- Uno schema XSD definisce:
 - gli elementi di un documento XML e la loro relazione gerarchica
 - gli eventuali attributi negli elementi
 - il numero e l'ordinamento degli elementi
 - gli eventuali elementi vuoti
 - il tipo di dati contenuto negli elementi e negli attributi
 - i valori predefiniti o costanti del contenuto degli elementi e degli attributi
- Il documento XSD che definisce la struttura di un documento XML va dichiarato nel prologo

```
xmlns:xsi="http://www.w3.org/XMLSchema-instance"  
xsi:noNamespaceSchemaLocation="..."
```

La struttura di un documento XML: XSD (3)

- Nel seguente documento XML si assume che lo schema sia stato salvato nel file libro.xsd

```
<?xml version="1.0" ?>
<libro
  xmlns:xsi="http://www.w3.org/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="libro.xsd">
  <autore>Stefano Benni</autore>
  <titolo>Terra!</titolo>
  <editore>Feltrinelli</editore>
  <anno>1983</anno>
</libro>
```

La struttura di un documento XML: XSD (4)

- In XSD possiamo definire elementi **semplici** e **complessi**
- Un elemento semplice è un elemento XML che può contenere **solo testo** (stringhe di caratteri, numeri, date, orari, ...), non può includere attributi o altri elementi

```
<xs:element name="..." type="..."/>
```

I tipi più comuni per un elemento semplice sono riportati nella TABELLA 4:

TABELLA 4

xs:string	Stringhe di caratteri.
xs:decimal	Valori numerici non necessariamente interi.
xs:integer	Valori numerici interi.
xs:date	Data nel formato aaaa-mm-gg (a: anno, m: mese, g: giorno).
xs:time	Ora nel formato oo-mm-ss (o: ora, m: minuti, s: secondi).

La struttura di un documento XML: XSD (5)

Esempi di definizione di elementi semplici

```
<xs:element name="nome" type="xs:string"/>
<xs:element name="dataNascita" type="xs:date"/>
<xs:element name="età" type="xs:integer"/>
```

e di relativi elementi XML validi

```
<nome>Walt Disney</nome>
<dataNascita>1901-12-05</dataNascita>
<età>65</età>
```

e invalidi:

```
<nome/>
<dataNascita>5/12/1901</dataNascita>
<età>sessantacinque</età>
```

La struttura di un documento XML: XSD (6)

- Un elemento semplice può avere un valore **predefinito** o un valore **costante**

Esempi di definizione di elementi semplici con valori predefiniti e costanti:

```
<xs:element name="colore" type="xs:string" default="rosso"/>  
<xs:element name="piGreco" type="xs:decimal" fixed="3.14159"/>
```

e di relativi elementi XML validi:

```
<colore>verde</colore>  
<colore/>  
<piGreco>3.14159</piGreco>
```

e invalidi:

```
<piGreco>3.1416</piGreco>
```

La struttura di un documento XML: XSD (7)

- Un elemento semplice può avere delle **restrizioni**

```
<xs:element name="quantità">
```

```
  <xs:simpleType>
```

```
    <xs:restriction base="xs:integer">
```

```
      <xs:minExclusive value="0"/>
```

```
      <xs:maxInclusive value="10"/>
```

```
    </xs:restriction>
```

```
  </xs:simpleType>
```

```
</xs:element>
```

```
<xs:element name="prezzo">
```

```
  <xs:simpleType>
```

```
    <xs:restriction base="xs:decimal">
```

```
      <xs:fractionDigits value="2"/>
```

```
    </xs:restriction>
```

```
  </xs:simpleType>
```

```
</xs:element>
```

e di relativi elementi XML validi

```
<quantità>5</quantità>
```

```
<quantità>10</quantità>
```

```
<prezzo>0.99</prezzo>
```

```
<prezzo>99</prezzo>
```

e invalidi:

```
<quantità>0</quantità>
```

```
<quantità>100</quantità>
```

```
<prezzo>1.999</prezzo>
```

La struttura di un documento XML: XSD (8)

- Un elemento semplice può avere inoltre delle restrizioni per definire un modello di stringa che deve essere rispettato
- `xs:pattern` consente di specificare una espressione regolare che la stringa deve soddisfare
- Esempio: supponiamo di voler creare una restrizione per il codice postale (che è composto da 5 numeri)

```
<xs:simpleType name="CodicePostaleType">  
  <xs:restriction base="xs:string">  
    <xs:pattern value="\d{5}"/>  
  </xs:restriction>  
</xs:simpleType>
```

In questo caso, il valore dell'elemento che utilizza questo tipo dovrà essere una stringa composta esattamente da 5 numeri (ad esempio: "12345").

La struttura di un documento XML: XSD (9)

- Un elemento **complesso** è:
 - un elemento vuoto, eventualmente con attributi
 - un elemento che contiene altri elementi

```
<xs:complexType name="persona">
  <xs:sequence>
    <xs:element name="cognome" type="xs:string"/>
    <xs:element name="nome" type="xs:string"/>
    <xs:element name="dataNascita" type="xs:date"/>
    <xs:element name="luogoNascita" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="docente" type="persona"/>
<xs:element name="studente" type="persona"/>
```


La struttura di un documento XML: XSD (10)

- un elemento che contiene altri elementi (anche con restrizioni)

```
<xs:element name="indirizzo">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="via" type="xs:string"/>
      <xs:element name="numero">
        <xs:simpleType>
          <xs:restriction base="xs:integer">
            <xs:minInclusive value="1"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="CAP">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:length value="5"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="città" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

valida elementi XML come questo

```
<indirizzo>
  <via>Via della Venezia</via>
  <numero>1</numero>
  <CAP>57100</CAP>
  <città>Livorno</città>
</indirizzo>
```

La struttura di un documento XML: XSD (11)

- un elemento che contiene solo testo (quindi elemento semplice), ma che prevede uno o più attributi

```
<xs:element name="misura">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:decimal">
        <xs:attribute name="unità" type="xs:string" use="required"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

valida elementi XML come questi

```
<misura unità="cm">100</misura>
<misura unità="mm">0.001</misura>
```

ma non come quello che segue

```
<misura>10</misura>
```

La struttura di un documento XML: XSD (12)

- un elemento che contiene sia testo che altri elementi

```
<xs:element name="messaggio">
  <xs:complexType mixed="true">
    <xs:sequence>
      <xs:element name="codice" type="xs:string"/>
      <xs:element name="data" type="xs:date"/>
      <xs:element name="ora" type="xs:time"/>
      <xs:element name="luogo" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

permette di validare elementi XML come quello che segue:

```
<messaggio>
La spedizione <codice>0123456789</codice> è stata consegnata il giorno
<data>2021-10-22</data> alle ore <ora>15:30</ora> a <luogo>Livorno</luogo>
</messaggio>
```

Differenza tra attributi ed elementi (1)

- I dati possono essere rappresentati attraverso gli **attributi** oppure attraverso gli **elementi**
- Un elemento è tutto ciò che va dal tag iniziale al tag finale e può avere diversi tipi di contenuto (anche altri elementi o essere vuoto)
- Un attributo è tutto ciò che è interno a un tag racchiuso tra singoli o doppi apici
- Usiamo gli elementi quando dobbiamo rappresentare dati che rappresentano delle **entità** o strutture gerarchiche, concetti che necessitano di ulteriori dettagli
- Usiamo gli attributi quando dobbiamo rappresentare dati su un elemento che sono **proprietà** o **metadati** (dati che descrivono altri dati) o informazioni aggiuntive (come un identificatore)
- **Seguite la logica che usate nel modello ER nei DB relazionali!**

Differenza tra attributi ed elementi (2)

```
<persone>
  <!-- caso 1: sesso come attributo-->
  <persona sesso="f">
    <nome>Anna</nome>
    <cognome>Rossi</cognome>
    <nazione>Italia</nazione>
  </persona>
  <!-- caso 2: sesso come elemento-->
  <persona>
    <sesso>f</sesso>
    <nome>Anna</nome>
    <cognome>Rossi</cognome>
    <nazione>Italia</nazione>
  </persona>
</persone>
```

Namespace

- All'interno dello stesso documento si potrebbero avere elementi o attributi con significati **diversi**
- Per evitare problemi di **omonimie** si usano i namespace
- I namespace vengono utilizzati anche nei linguaggi di programmazione, sono **contenitori logici** (li ritroviamo usando *namespace* in C#, *package* in Java, *import* in Python, ...)
- In XML per definire un namespace si usa la seguente sintassi:

```
NamespacePrefix: xmlns="NamespaceURI"
```

```
<?xml version="1.0" encoding="ISO-8859-1"?>
- <lb:libro xmlns:lb="mioSito.com/libri">
  <lb:titolo>Libro di informatica</lb:titolo>
  - <ca:capitolo xmlns:ca="mioSito.com/capitoli">
    <ca:titolo>Documento in stile XML</ca:titolo>
    <ca:testo>testo testo testo</ca:testo>
  </ca:capitolo>
</lb:libro>
```

Attributi globali

- In XML, ci sono vari attributi prefissati con `xml:` che sono definiti dal W3C e sono `globali`, cioè possono essere utilizzati in qualsiasi elemento XML
- Questi attributi sono `riservati` e hanno uno scopo specifico nell'ambito della `gestione` e del `trattamento` dei documenti XML
- Ad es. XML prevede un attributo predefinito speciale denominato `xml:lang` per indicare la lingua del testo contenuto in un elemento e nei suoi figli (il suo valore è il codice della lingua che può essere un codice ISO 639-1)
- Ad es. XML prevede un attributo predefinito speciale denominato `xml:id` per assegnare un identificatore univoco all'elemento

Il parsing di XML (1)

- Come viene analizzato un documento XML?
- Il linguaggio di programmazione che manipola un documento XML deve prevedere un **parser**, ovvero uno strumento che lo legge, lo interpreta e lo converte in una struttura che può essere facilmente manipolata e utilizzata: **un albero sintattico**
- Per evitare che il contenuto di un elemento sia analizzato dal parser e di conseguenza per permettere che al suo interno possa essere inserita qualsiasi sequenza di caratteri è possibile definirlo come CDATA (Character DATA)
- Ad es. è utile nel caso in cui si vogliano inserire all'interno di un elemento delle stringhe di codice o dell'XML o dell'HTML senza che questi vengano analizzati e quindi interpretati erroneamente
- Il testo va inserito tra **<![CDATA[** e **]]>**

Il parsing di XML (2)

```
<?xml version="1.0" ?>
<MESSAGE timestamp="07/10/2021 12:00:00">
  <from>Giorgio</from>
  <to>Fiorenzo</to>
  <text>
    <![CDATA[
      Ti invio un esempio di XML:
      <?xml version="1.0" ?>
      <users>
        <user>
          <username>Pippo</username>
          <email>pippo32@disney.com</email>
        </user>
        <user>
          <username>Pluto</username>
          <email>pluto30@disney.com</email>
        </user>
      </users>
    ]]>
  </text>
</MESSAGE>
```

Il parsing di HTML

- Come viene analizzato un documento HTML?
- Anche quando un browser carica una pagina web c'è un parser che analizza il codice HTML e lo converte in una struttura dati gerarchica
- La struttura risultante è il DOM (Document Object Model), un oggetto che rappresenta l'intero documento HTML e tutti i suoi elementi come nodi
- Il DOM viene utilizzato come interfaccia di programmazione, ovvero consente ai linguaggi di programmazione (come JavaScript) di accedere, manipolare e interagire dinamicamente con il contenuto e la struttura di una pagina web